

APIs & Authentication

الطالبة : نسمة عبدالقادر بن شهاب

المقدمة

تُعد واجهات برمجة التطبيقات (APIs) وأنظمة المصادقة (Authentication) من أهم التقنيات المستخدمة في تطوير التطبيقات الحديثة. حيث تسمح الـ APIs بالتواصل وتبادل البيانات بين الواجهة الأمامية (Frontend) والخلفية (Backend)، مما يساعد على بناء تطبيقات أكثر تنظيمًا وكفاءة. أما أنظمة المصادقة فتُستخدم للتحقق من هوية المستخدمين وحماية البيانات من الوصول غير المصرح به. وتُستخدم هذه التقنيات في العديد من الأنظمة الحديثة مثل مواقع التواصل الاجتماعي، التطبيقات البنكية، والمتاجر الإلكترونية.

ما هو المقصود بالـ API ؟

API أو واجهة برمجة التطبيقات هي مجموعة من القواعد والبروتوكولات التي تسمح لتطبيقات أو أكثر بالتواصل مع بعضها البعض. باختصار، يمكن اعتبار الـ API كجسر يربط بين برنامجك وخدمات أخرى دون الحاجة لمعرفة التفاصيل الداخلية لهذه الخدمات.

لماذا نُجد الـ API ؟

وُجد الـ API لتسهيل التواصل وتبادل البيانات بين الأنظمة والتطبيقات المختلفة بطريقة منظمة وآمنة. فهو يعمل كوسيط بين الواجهة الأمامية (Frontend) والخلفية (Backend)، مما يسمح للتطبيقات بإرسال الطلبات واستقبال البيانات دون الحاجة لمعرفة كيفية عمل النظام الداخلي بالكامل. كما يساعد الـ API على:

- تسهيل تطوير التطبيقات .
- توفير الوقت والجهد بإعادة استخدام الخدمات الجاهزة .
- ربط الأنظمة المختلفة مع بعضها .
- تنظيم عملية إرسال واستقبال البيانات .
- جعل التطبيقات أكثر مرونة وقابلية للتطوير .

كيفية عمل واجهات برمجة التطبيقات

عادة ما يتبع التفاعل بين الواجهة الأمامية والخلفية عبر واجهة برمجة التطبيقات نموذج طلب-استجابة.

تفاعل واجهة برمجة التطبيقات خطوة بخطوة:

1. الطلب: ترسل الواجهة الأمامية طلب HTTP إلى الخادم عبر نقطة نهاية (API).

2. **المعالجة:** يقوم القسم الخلفي بمعالجة الطلب، وجلب البيانات أو التلاعب بها، وتحضير الرد.

3. **الرد:** يرسل الطرف الخلفي استجابة HTTP إلى الواجهة الأمامية، والتي تعرض البيانات للمستخدم.

ما هو REST ؟

REST اختصار لـ **Representational State Transfer**، وهو أسلوب أو طريقة تُستخدم لتصميم وبناء واجهات برمجة التطبيقات (APIs) بطريقة منظمة وسهلة الفهم. يعتمد REST على بروتوكول HTTP المستخدم في الإنترنت لإرسال واستقبال البيانات بين الـ Client والـ Server.

عندما يستخدم التطبيق REST API فإن التواصل يتم عن طريق إرسال طلبات (Requests) إلى الخادم، ثم يقوم الخادم بإرجاع ردود (Responses) تحتوي على البيانات المطلوبة أو نتيجة العملية.

يُعتبر REST من أكثر الأساليب استخدامًا في تطوير تطبيقات الويب والموبايل بسبب بساطته وسهولة التعامل معه.

لماذا وُجد REST ؟

وُجد REST لتنظيم عملية التواصل بين الأنظمة والتطبيقات المختلفة بطريقة موحدة وواضحة، حيث كانت الأنظمة سابقًا تستخدم طرقًا معقدة وغير منظمة لتبادل البيانات.

يساعد REST في:

- تسهيل التواصل بين الـ Frontend والـ Backend.
- جعل الـ APIs أسهل للفهم والتطوير.
- تنظيم طريقة إرسال واستقبال البيانات.
- دعم تطبيقات الويب والموبايل والأنظمة المختلفة.
- تقليل التعقيد في بناء الأنظمة.

ما هو الـ Endpoint ؟

الـ Endpoint هو الرابط أو المسار الذي يتم إرسال الطلب إليه داخل الـ API لتنفيذ عملية معينة أو الوصول إلى بيانات محددة.

كل Endpoint يكون مسؤولًا عن وظيفة معينة داخل النظام.

مثال:

هذا ال Endpoint قد يُستخدم للحصول على بيانات المستخدمين.

كيف يعمل REST مع ال Endpoints ؟

يقوم ال Client (مثل تطبيق الويب أو الهاتف) بإرسال Request إلى Endpoint معين باستخدام نوع محدد من الطلبات مثل GET أو POST ، ثم يقوم ال Server بمعالجة الطلب وإرجاع Response مناسب.

أنواع الطلبات الأساسية في REST

1. **GET** : يُستخدم لجلب أو قراءة البيانات من الخادم دون تعديلها. أي أنه يُستخدم فقط للحصول على المعلومات.
2. **POST** : يُستخدم لإرسال بيانات جديدة إلى الخادم وإنشائها داخل قاعدة البيانات.
3. **PUT** : يُستخدم لتحديث أو تعديل بيانات موجودة مسبقاً داخل النظام.
4. **DELETE** : يُستخدم لحذف البيانات من قاعدة البيانات.

ما هو ال Request ؟

ال Request هو الطلب الذي يرسله ال Client (مثل المتصفح أو تطبيق الهاتف) إلى ال Server لطلب بيانات أو تنفيذ عملية معينة.

عندما يقوم المستخدم بأي إجراء داخل التطبيق مثل تسجيل الدخول أو عرض المنتجات، يقوم التطبيق بإرسال Request إلى الخادم لمعالجة هذا الطلب.

مكونات ال Request

1. **Method** : وهو نوع الطلب المستخدم مثل GET
2. **URL** أو **Endpoint** : وهو الرابط الذي يتم إرسال الطلب إليه.
3. **Headers** : تحتوي على معلومات إضافية عن الطلب مثل:
 - نوع البيانات.
 - معلومات التوثيق. Authentication.
 - التوكن. Token.
4. **Body** : يحتوي على البيانات المرسله إلى الخادم، ويُستخدم غالباً مع POST و PUT.

ما هو ال Response ؟

ال Response هو الرد الذي يرسله ال Server بعد استلام ال Request ومعالجته.

مكونات الResponse

1. **Status Code**: وهو رقم يوضح حالة الطلب.
2. **Headers**: تحتوي على معلومات إضافية عن الرد.
3. **Body**: يحتوي على البيانات التي يعيدها الخادم.

Authentication vs Authorization

تُعد عمليتا Authentication و Authorization من أهم المفاهيم الأمنية في تطوير التطبيقات والأنظمة الحديثة، وغالبًا ما يتم استخدامهما معًا لحماية البيانات والتحكم في وصول المستخدمين إلى النظام. على الرغم من أن المفهومين مرتبطان بالأمان، إلا أن لكل منهما وظيفة مختلفة.

أولاً ما هو Authentication ؟

Authentication يعني **التحقق من هوية المستخدم**، أي التأكد من أن الشخص الذي يحاول الدخول إلى النظام هو بالفعل صاحب الحساب الحقيقي. تحدث هذه العملية عادة عند تسجيل الدخول إلى التطبيق أو الموقع.

كيف يعمل Authentication ؟

عندما يقوم المستخدم بإدخال:

- البريد الإلكتروني أو اسم المستخدم.
- كلمة المرور.

يقوم النظام بمقارنة هذه البيانات مع البيانات المخزنة في قاعدة البيانات، وإذا كانت صحيحة يتم السماح للمستخدم بالدخول.

لماذا وُجد Authentication ؟

وُجد لحماية الحسابات والبيانات ومنع الأشخاص غير المصرح لهم من الدخول إلى النظام. بدون Authentication يمكن لأي شخص الوصول إلى حسابات المستخدمين وبياناتهم الخاصة.

ثانيًا ما هو Authorization ؟

Authorization يعني **تحديد الصلاحيات أو الأدونات التي يمتلكها المستخدم بعد تسجيل الدخول**.

أي أنه يحدد:

- ما الذي يستطيع المستخدم الوصول إليه؟
- وما العمليات المسموح له بتنفيذها؟

كيف يعمل Authorization ؟

بعد أن يتأكد النظام من هوية المستخدم عبر Authentication ، يبدأ بفحص صلاحياته.

فمثلاً:

- المدير يمتلك صلاحيات كاملة.
- الموظف يمتلك صلاحيات محدودة.
- المستخدم العادي يمتلك صلاحيات بسيطة.

لماذا وُجد Authorization ؟

وُجد للتحكم في وصول المستخدمين إلى أجزاء النظام المختلفة ومنعهم من تنفيذ عمليات غير مسموحة لهم.

الفرق بين Authentication و Authorization

Authentication	Authorization
التحقق من هوية المستخدم	تحديد صلاحيات المستخدم
يحدث أولاً	يحدث بعد تسجيل الدخول
يجيب على سؤال: من أنت؟	يجيب على سؤال: ماذا يمكنك أن تفعل؟
يعتمد على كلمة المرور أو وسائل التحقق	يعتمد على الأدوار والصلاحيات
هدفه حماية الحسابات	هدفه حماية الموارد والبيانات

العلاقة بينهما

عادةً لا يمكن تنفيذ Authorization قبل Authentication.

أي أن النظام:

1. يتحقق أولاً من هوية المستخدم.

2. ثم يحدد صلاحياته داخل النظام.

Basic Auth & Tokens

تُستخدم أنظمة التوثيق Authentication لحماية التطبيقات والتأكد من هوية المستخدمين قبل السماح لهم بالوصول إلى النظام. ومن أشهر الطرق المستخدمة في تسجيل الدخول: Basic Authentication و Tokens مثل JWT.

أولاً ما هو Basic Authentication :

هو أسلوب بسيط للتحقق من هوية المستخدم، حيث يقوم المستخدم بإرسال:

- اسم المستخدم أو البريد الإلكتروني.
- كلمة المرور.

مع كل Request يتم إرساله إلى الخادم.

عادة يتم إرسال هذه البيانات داخل الـ Headers بعد تحويلها إلى صيغة مشفرة بسيطة باستخدام Base64.

كيف يعمل Basic Authentication ؟

1. يقوم المستخدم بإدخال البريد الإلكتروني وكلمة المرور.
2. يقوم التطبيق بإرسال البيانات إلى الخادم.
3. يتحقق الخادم من صحة البيانات.
4. إذا كانت صحيحة يسمح النظام بالوصول.

مثال على Basic Auth

Authorization: Basic dXNlcjpwYXNzd29yZA==

القيمة السابقة عبارة عن اسم المستخدم وكلمة المرور بعد تحويلها إلى Base64.

لماذا وُجد Basic Authentication ؟

وُجد لتوفير طريقة بسيطة وسريعة لتسجيل الدخول والتحقق من المستخدمين.

مشاكل Basic Authentication

رغم بساطته إلا أنه يحتوي على بعض المشاكل الأمنية، أهمها:

- إرسال كلمة المرور مع كل Request.
- يمكن سرقة البيانات إذا لم يتم استخدام HTTPS.
- أقل أمانًا مقارنة بالأنظمة الحديثة.

لذلك أصبح استخدامه محدودًا في التطبيقات الحديثة.

ثانيًا ما هو الـ Token ؟

الـ Token هو رمز أو مفتاح يتم إنشاؤه بعد نجاح عملية تسجيل الدخول، ويُستخدم لإثبات هوية المستخدم بدل إرسال كلمة المرور في كل مرة.

لماذا وُجد الـ Token ؟

وُجد لحل مشاكل الأمان الموجودة في Basic Authentication ، ولجعل عملية تسجيل الدخول أكثر أمانًا وكفاءة.

كيف يعمل تسجيل الدخول باستخدام Tokens ؟

الخطوات بالتفصيل:

1. إدخال بيانات تسجيل الدخول

يقوم المستخدم بإدخال:

- البريد الإلكتروني.
- كلمة المرور.

2. إرسال Request إلى الخادم

POST /login

3. التحقق من البيانات

يقوم الخادم بمقارنة البيانات مع قاعدة البيانات.

4. إنشاء Token

إذا كانت البيانات صحيحة يقوم الخادم بإنشاء Token خاص بالمستخدم.

5. إرسال الـ Token للمستخدم

مثال على: Response

```
{  
  "token": "abc123xyz"  
}
```

6. استخدام الـ Token لاحقاً

في كل Request جديد يرسل المستخدم الـ Token بدل كلمة المرور.

مثال:

Authorization: Bearer abc123xyz

فوائد استخدام Tokens

- أكثر أماناً.
- لا حاجة لإرسال كلمة المرور باستمرار.
- مناسب لتطبيقات الويب والموبايل.
- يسهل إدارة الجلسات. Sessions
- يقلل خطر سرقة كلمات المرور.

ما هو JWT ؟

JWT اختصار لـ: JSON Web Token

وهو نوع مشهور جداً من الـ Tokens يُستخدم في التطبيقات الحديثة.

لماذا يُستخدم JWT ؟ يُستخدم JWT لأنه:

- سريع.
- آمن نسبياً.
- سهل الاستخدام.
- لا يحتاج إلى تخزين Session داخل الخادم.
- مناسب للأنظمة الحديثة والـ APIs.

مكونات JWT

يتكون JWT من 3 أجزاء رئيسية:

1. Header

يحتوي على:

- نوع الـ Token.
- خوارزمية التشفير.

2. Payload

يحتوي على معلومات المستخدم مثل:

- ID
- Email
- Role

3. Signature

يُستخدم للتحقق من صحة الـ Token ومنع التلاعب به.

شكل JWT

يظهر غالبًا بهذا الشكل:

xxxxx.yyyyy.zzzzz

كل جزء مفصول بنقطة.

كيف يعمل JWT ؟

الخطوات:

1. يسجل المستخدم الدخول.
2. يتحقق الخادم من البيانات.
3. يتم إنشاء JWT يحتوي على بيانات المستخدم.
4. يُرسل JWT إلى المستخدم.
5. يرسل المستخدم الـ JWT مع كل Request.
6. يتحقق الخادم من صحة الـ JWT ثم يسمح بالوصول.

الفرق بين Basic Auth و Tokens

Basic Auth	Tokens
يرسل Token بدل كلمة المرور	يرسل كلمة المرور مع كل Request
أكثر أمانًا	أقل أمانًا
حديث وأكثر استخدامًا	بسيط لكنه قديم
مناسب لل APIs الحديثة	يعتمد على Sessions غالبًا

Password Hashing

Password Hashing هو عملية تحويل كلمة المرور إلى قيمة مشفرة وثابتة تُسمى Hash باستخدام خوارزمية خاصة، بحيث لا يمكن قراءة كلمة المرور الأصلية بسهولة. بدلاً من تخزين كلمة المرور الحقيقية داخل قاعدة البيانات، يتم تخزين ال Hash فقط لحماية حسابات المستخدمين.

ماذا يعني Hashing ؟

ال Hashing هو عملية رياضية تقوم بتحويل البيانات إلى سلسلة من الأحرف والأرقام ذات طول ثابت.

مثال:

كلمة المرور:

123456

قد تتحول إلى:

e10adc3949ba59abbe56e057f20f883e

هذه القيمة تُسمى Hash.

لماذا لا نخزن كلمات المرور كنص عادي؟

إذا تم تخزين كلمات المرور كنص عادي (Plain Text) وتم اختراق قاعدة البيانات، سيتمكن المخترق من رؤية جميع كلمات المرور الحقيقية للمستخدمين.

وهذا قد يؤدي إلى:

- اختراق الحسابات.
 - سرقة البيانات.
 - استخدام نفس كلمات المرور في مواقع أخرى.
 - فقدان ثقة المستخدمين بالنظام.
- لذلك لا يتم تخزين كلمات المرور الحقيقية أبدًا في الأنظمة الآمنة.

كيف يعمل Password Hashing ؟

1. إنشاء الحساب

عندما يقوم المستخدم بإنشاء كلمة مرور:

mypassword123

يقوم النظام بتحويلها إلى Hash باستخدام خوارزمية Hashing.

2. تخزين الـ Hash

بدل تخزين كلمة المرور الأصلية، يتم تخزين قيمة الـ Hash فقط داخل قاعدة البيانات.

3. تسجيل الدخول

عندما يقوم المستخدم بإدخال كلمة المرور مرة أخرى:

- يقوم النظام بتحويل الكلمة المدخلة إلى Hash.
- ثم يقارنها بالـ Hash المخزن في قاعدة البيانات.

إذا تطابقت القيمتان يتم تسجيل الدخول بنجاح.

لماذا يُعتبر Hashing آمناً؟

لأن عملية الـ Hashing غالبًا تكون:

- One-Way Function

أي لا يمكن استرجاع كلمة المرور الأصلية بسهولة من الـ Hash.

حتى لو حصل المخترق على قاعدة البيانات، فلن يرى كلمات المرور الحقيقية مباشرة.

أشهر خوارزميات الـ Hashing

1. bcrypt

من أكثر الخوارزميات استخدامًا وأمانًا.

2. Argon2

خوارزمية حديثة وآمنة جدًا.

3. SHA-256

تُستخدم في بعض الأنظمة، لكنها غالبًا تحتاج تقنيات إضافية للحماية.

ما هو Salt ؟

ال Salt هو قيمة عشوائية تُضاف إلى كلمة المرور قبل عملية ال Hashing لزيادة الأمان ومنع هجمات التخمين.

مثال على Salt

بدل تحويل:

123456

يتم دمجها مع Salt ثم تحويلها إلى Hash.

هذا يجعل حتى كلمات المرور المتشابهة تنتج قيم Hash مختلفة.

الخاتمة

في الختام، تُعد APIs وأنظمة Authentication من الأساسيات المهمة في تطوير التطبيقات الحديثة، حيث تساعد ال APIs على تسهيل التواصل وتبادل البيانات بين ال Frontend وال Backend بطريقة منظمة وفعالة. كما تساهم تقنيات Authentication و Authorization في حماية الأنظمة والتحقق من هوية المستخدمين وتحديد صلاحياتهم داخل التطبيق.

بالإضافة إلى ذلك، تُستخدم Tokens و JWT للتوفير طرق أكثر أمانًا لإدارة تسجيل الدخول، بينما يساعد Password Hashing على حماية كلمات المرور ومنع تخزينها بشكل مكشوف داخل قواعد البيانات. لذلك تُعتبر هذه التقنيات ضرورية لبناء تطبيقات آمنة، سريعة، وموثوقة تُستخدم في مختلف الأنظمة الحديثة مثل التطبيقات البنكية، المتاجر الإلكترونية، ومنصات التواصل الاجتماعي.